

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



CSC10014 - TƯ DUY TÍNH TOÁN
HKII: 2025 - 2026

HỆ THỐNG LƯU TRỮ DU LỊCH THÔNG MINH

Lớp : CQ2024/6
Nhóm thực hiện : Nhóm 12
Giáo viên hướng dẫn : Hồ Tuấn Thanh
GVHDTH : Mai Anh Tuấn
Trợ giảng : Phạm Nguyễn Sơn Tùng

Thành Viên:

24120033: Đào Tiến Đạt.
24120040: Dương Hoài Đức.
24120304: Trần Thị Bích Hạnh.
24120310: Nguyễn Đình Hiếu.
24120483: Hoàng Văn Trường.

THÀNH PHỐ HỒ CHÍ MINH, NGÀY 18/6/2026

Lời cảm ơn

Trong quá trình thực hiện đề án môn học, nhóm chúng em đã nhận được rất nhiều sự hỗ trợ và hướng dẫn quý báu từ các thầy cô và bạn bè.

Trước hết, chúng em xin gửi lời cảm ơn chân thành đến thầy Hồ Tuấn Thanh – giảng viên phụ trách học phần, đã cung cấp những kiến thức nền tảng cũng như tạo điều kiện để chúng em có cơ hội tiếp cận với các công nghệ và quy trình phát triển ứng dụng thực tế. Chúng em cũng xin chân thành cảm ơn thầy Mai Anh Tuấn – giảng viên hướng dẫn thực hành cùng các thầy cô trợ giảng đã tận tình hỗ trợ, giải đáp các thắc mắc trong quá trình học tập và thực hiện đề án.

Thông qua học phần này, các thành viên trong nhóm không chỉ tích lũy được những kiến thức chuyên môn quý báu mà còn rèn luyện được tư duy giải quyết vấn đề, kỹ năng làm việc nhóm và khả năng vận dụng kiến thức vào các bài toán thực tế. Những kinh nghiệm thu được trong quá trình thực hiện đề tài sẽ là nền tảng quan trọng cho việc học tập và nghiên cứu trong tương lai.

Mặc dù đã nỗ lực hoàn thành đề án với tinh thần học hỏi và trách nhiệm cao nhất, song do kiến thức và kinh nghiệm thực tiễn còn hạn chế nên bài làm khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp và nhận xét quý báu từ Thầy/Cô để có thể tiếp tục hoàn thiện kiến thức và kỹ năng của mình.

Cuối cùng, chúng em xin kính chúc quý Thầy/Cô luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công trong sự nghiệp giảng dạy và nghiên cứu.

Các thành viên của nhóm 12

Mục lục

1	Group member	3
2	Idea	4
2.1	Đặt vấn đề	4
2.2	Tầm nhìn ứng dụng	4
2.3	Người dùng mục tiêu	4
3	Problem Analysis & Decomposition	6
3.1	Sử dụng phương pháp User Journey	6
3.2	Từ bài toán thực tế đến bài toán tính toán	7
3.3	Ràng buộc, giả định và tiêu chí đánh giá	7
4	System Overview	9
4.1	List of Core Features	9
4.2	Explanation of Core Features	9
4.3	Innovation Highlights	10
5	Pattern Recognition	11
6	Abstraction	12
7	System/Algorithm Design	13
7.1	Biểu diễn Bài toán	13
7.2	Tổng quan Công nghệ	13
7.3	Mô hình C4	14
7.3.1	Cấp 1 — System Context	14
7.3.2	Cấp 2 — Container	14
7.3.3	Cấp 3 — Component	15
7.3.4	Cấp 4 — Code (Chi tiết Module AI)	15
7.4	Thiết kế Giải thuật theo Chức năng	15
7.4.1	Điều phối AI	15
7.4.2	Tìm kiếm Lai (Hybrid Search)	16
7.4.3	Gợi ý Xếp hạng	17
7.4.4	Đường ống RAG	18
7.4.5	Các Giải thuật Bỏ trợ	18
7.5	Đánh giá Hiệu năng và Khả năng Mở rộng	18
8	Implementation	19
8.1	Từ Mô hình Thiết kế đến Phân rã Module	19
8.2	Di trú Kiến trúc: Microservices → Modular Monolith	19
8.3	Các Thách thức và Giải pháp	19
8.3.1	Tính phi tất định của LLM	19
8.3.2	Streaming SSE qua POST	20
8.3.3	Tích hợp Đa Nhà cung cấp Bản đồ	20
8.3.4	Vector Embedding và Prisma	20
8.3.5	Xác thực Firebase	20
8.4	Các Thách thức Khác	21

8.5	Kết luận	21
9	Testing	22
9.1	Chiến lược Kiểm thử	22
9.2	Phạm vi Kiểm thử	22
9.2.1	Frontend — Giao diện và Tương tác	22
9.2.2	Backend — Xử lý Nghiệp vụ và Tích hợp	22
9.2.3	Các Tình huống và Giả lập Đặc thù	23
9.3	Kết quả Kiểm thử	23
9.4	Cải tiến Sau Kiểm thử	24
9.5	Hướng Phát triển	25
10	Demo	26
11	Deployment	27
12	Logbook: Plan and Actual Workload	28
13	AI usage and declaration	29
14	Kết luận và hướng phát triển	30
14.1	Kết luận	30
14.2	Những kiến thức và kỹ năng đạt được	30
14.3	Hướng phát triển trong tương lai	31

1 Group member

- **Thành viên 1:**
Tên: Đào Tiến Đạt
MSSV: 24120033
Email:
Vai trò/Nhiệm vụ:
- **Thành viên 2:**
Tên: Dương Hoài Đức
MSSV: 24120040
Email:
Vai Trò/Nhiệm vụ:
- **Thành viên 3:**
Tên: Trần Thị Bích Hạnh
MSSV: 24120304
Email: 24120304@student.hcmus.ed.vn
Vai trò/Nhiệm vụ:
- **Thành Viên 4:**
Tên: Nguyễn Đình Hiếu
MSSV: 24120310
Email:
Vai trò/Nhiệm vụ:
- **Thành viên 5:**
Tên: Hoàng Văn Trường
MSSV: 24120483
Email:
Vai trò/Nhiệm vụ:

2 Idea

2.1 Đặt vấn đề

Hiện nay, việc tìm kiếm một nơi lưu trú phù hợp khi đi du lịch không hề đơn giản. Người dùng thường phải cân nhắc nhiều yếu tố như giá cả, vị trí, tiện nghi, chất lượng dịch vụ và các đánh giá từ những khách hàng trước đó. Tuy nhiên, quá trình này vẫn còn gặp nhiều khó khăn do các nguyên nhân sau:

- **Thông tin phân tán trên nhiều nền tảng:** Thông tin về khách sạn, homestay hay resort thường được đăng tải trên nhiều website và ứng dụng khác nhau. Người dùng phải mất nhiều thời gian để tìm kiếm, so sánh giá cả, tiện nghi và đánh giá trước khi đưa ra quyết định.
- **Khó tìm được địa điểm phù hợp với nhu cầu cá nhân:** Mỗi người dùng có những tiêu chí khác nhau về nơi lưu trú như ngân sách, vị trí, loại hình chỗ ở hoặc các tiện ích đi kèm. Tuy nhiên, các công cụ tìm kiếm truyền thống chủ yếu dựa trên bộ lọc cố định nên chưa hỗ trợ tốt cho việc tìm kiếm theo nhu cầu tự nhiên của người dùng.

Hệ quả: Người dùng phải tốn nhiều thời gian để thu thập và xử lý thông tin nhưng vẫn có thể gặp khó khăn trong việc lựa chọn nơi lưu trú phù hợp nhất với nhu cầu của mình.

Giải pháp: Nhóm đề xuất xây dựng một hệ thống tìm kiếm và gợi ý lưu trú thông minh, cho phép người dùng mô tả nhu cầu bằng ngôn ngữ tự nhiên. Hệ thống sẽ phân tích yêu cầu, tìm kiếm các địa điểm phù hợp, thực hiện đánh giá và xếp hạng, sau đó đưa ra những gợi ý trực quan và dễ hiểu nhằm hỗ trợ người dùng đưa ra quyết định nhanh chóng và hiệu quả hơn.

2.2 Tầm nhìn ứng dụng

Tầm nhìn của dự án là xây dựng một nguyên mẫu thử nghiệm nhằm chứng minh tính khả thi của việc kết hợp trí tuệ nhân tạo vào quy trình tìm kiếm địa điểm lưu trú. Thay vì định hướng thành một nền tảng thương mại hoàn chỉnh ngay từ đầu, dự án tập trung vào việc định hình lại cách người dùng tương tác với dữ liệu.

Thông qua việc mô phỏng một môi trường tích hợp thông tin đa nguồn và ứng dụng kỹ thuật RAG (Retrieval-Augmented Generation), nguyên mẫu này kỳ vọng sẽ tạo ra một tiền đề (proof of concept) cho các hệ thống tìm kiếm trong tương lai. Ở đó, AI đóng vai trò như một tác tử điều phối trung gian, biến những truy vấn ngôn ngữ tự nhiên mơ hồ của người dùng thành các kết quả trực quan, đáng tin cậy và mang tính cá nhân hóa cao.

2.3 Người dùng mục tiêu

Smacco tập trung phục vụ ba nhóm đối tượng chính:

- **Khách du lịch tự túc:** Những người cần tìm kiếm thông tin chi tiết, so sánh giá và chất lượng một cách nhanh chóng để tối ưu hóa ngân sách và trải nghiệm.

- **Người dùng đang lưu trú (On-site Users):** Những người muốn đóng góp thông tin thực tế, hình ảnh tại chỗ và hỗ trợ cộng đồng thông qua việc trả lời các câu hỏi thực tế.
- **Người dùng cần sự hỗ trợ cá nhân hóa:** Những người ưu tiên việc sử dụng chatbot thông minh để tìm kiếm theo nhu cầu đặc biệt thay vì tự tay lọc qua hàng trăm kết quả.

3 Problem Analysis & Decomposition

3.1 Sử dụng phương pháp User Journey

Để phân rã và thiết kế hệ thống bám sát thực tiễn, nhóm đã đặt mình vào vị trí của người dùng (User Journey) và đặt ra chuỗi câu hỏi dẫn dắt như sau:

- **Người dùng sẽ làm gì với ứng dụng của chúng tôi?**
→ Người dùng cần tìm địa điểm lưu trú phù hợp với nhu cầu cá nhân.
- **Người dùng có thể tìm kiếm địa điểm lưu trú như thế nào?**
→ Cần thiết kế giao diện tìm kiếm hoặc cung cấp phương thức tìm kiếm linh hoạt (như tìm trên bản đồ, lọc theo tiêu chí, tìm bằng từ khóa).
- **Vậy làm sao để phương thức tìm kiếm có thể cho ra kết quả tốt?**
→ Chúng ta sẽ đánh giá trên các tiêu chí cụ thể; như vậy chúng ta cần trích xuất ý định thực sự của người dùng từ các yêu cầu thô.
- **Vậy việc trích xuất ý định nên được thực hiện thế nào?**
→ Có thể cho phép người dùng chủ động chọn qua bộ lọc, hoặc thông qua một tác tử AI được cài đặt để phân tích ngôn ngữ tự nhiên.
- **Nếu người dùng sau khi đã có các địa điểm phù hợp với nhu cầu của họ, làm cách nào để hệ thống có thể giúp người dùng ra quyết định cuối cùng?**
→ Có thể cài đặt tính năng giúp người dùng so sánh các địa điểm, qua đó hỗ trợ ra quyết định cuối cùng một cách khách quan.
- **Vậy nếu người dùng muốn tìm hiểu sâu hơn về một địa điểm thì sao?**
→ Cài đặt tính năng AI đưa ra phân tích chi tiết về một địa điểm cụ thể, cho phép người dùng lựa chọn các tiêu chí để phân tích (ví dụ: wifi, bữa sáng, thái độ phục vụ).
- **Việc sử dụng LLM có thể không đạt được độ chính xác cao, liệu nguồn thông tin nào uy tín hơn không?**
→ Sử dụng nguồn thông tin từ chính người dùng. Có thể áp dụng chính sách "người dùng onsite" (những người đang thực sự có mặt tại địa điểm) để tăng độ tin cậy.
- **Vậy với các tính năng như trên, có thể tối giản giao diện để người dùng có thể tương tác nhanh chóng và dễ dàng, phù hợp với nhiều người dùng ở các độ tuổi, trình độ khác nhau hay không?**
→ Sử dụng LLM đóng vai trò điều phối trung gian. Người dùng sẽ tương tác trực tiếp với LLM; sau đó, LLM sẽ trích xuất ý định người dùng và tiến hành các yêu cầu một cách tự động dưới nền.

Kết luận:

Thông qua việc đặt câu hỏi với vai trò là người dùng, nhóm thống nhất ý tưởng xây dựng một hệ thống hỗ trợ người dùng tìm kiếm địa điểm lưu trú với sự hỗ trợ thông minh của AI với vai trò là người hướng dẫn, điều phối. Hệ thống sẽ có các chức năng chính bao gồm:

1. **Tìm kiếm thông minh (Hybrid Search):** Hỗ trợ tìm kiếm địa điểm qua các bộ lọc truyền thống kết hợp với khả năng phân tích ý định ngôn ngữ tự nhiên từ AI.
2. **So sánh địa điểm lưu trú:** Cung cấp bảng đối chiếu các tiện ích, giá cả và đánh giá để hỗ trợ người dùng ra quyết định cuối cùng.
3. **Phân tích và cung cấp Insight bằng AI:** Khai thác hàng ngàn bài đánh giá thực tế để AI tóm tắt và trả lời các câu hỏi chuyên sâu về một địa điểm cụ thể.
4. **Cộng đồng Hỏi-Đáp và Check-in:** Nền tảng chia sẻ thông tin từ những người dùng thực tế, đặc biệt ưu tiên phản hồi từ người dùng đang "On-site" để đảm bảo tính uy tín của thông tin.

3.2 Từ bài toán thực tế đến bài toán tính toán

Từ việc nhận định được mục tiêu cụ thể, nhóm tiến hành phân rã các bài toán thực tế thành các bài toán tính toán cụ thể hơn như sau:

- **Phân tích ý định ngôn ngữ tự nhiên:** Chuyển đổi các câu truy vấn tự do của người dùng thành các tham số truy vấn có cấu trúc nhằm kích hoạt workflow cụ thể.
- **Tìm kiếm và Tổng hợp Dữ liệu:** Xây dựng cơ chế truy xuất đồng thời từ cơ sở dữ liệu nội bộ và các nhà cung cấp bên thứ ba, sau đó chuẩn hóa và loại bỏ dữ liệu trùng lặp.
- **Truy xuất thông tin ngữ nghĩa:** Số hóa các user reviews và dữ liệu hỏi đáp cộng đồng thành các vector embeddings. Điều này cho phép hệ thống thực hiện tìm kiếm theo ngữ nghĩa, giúp trợ lý AI đưa ra câu trả lời dựa trên trải nghiệm thực tế thay vì gặp ảo giác.
- **Tích hợp dữ liệu và hệ thống tính toán ngoài:** Xây dựng luồng kết nối API và đồng bộ hóa dữ liệu từ các hệ thống ngoại vi để đáp ứng nhu cầu truy xuất thông tin liên tục và đảm bảo tính nhất quán của dữ liệu.
- **Xây dựng khung làm việc cố định cho LLM:** Thiết lập các quy tắc, cấu trúc prompt và hệ thống workflow cứng nhằm kiểm soát hành vi của LLM, giúp AI hoạt động một cách nhất quán, chính xác và có thể dự đoán được.
- **Lập kế hoạch phát triển giao diện phần mềm, nâng cao tính dễ tiếp cận:** Phân tích và thiết kế trải nghiệm người dùng (UX/UI), đảm bảo tính trực quan để người dùng ở nhiều độ tuổi và trình độ công nghệ khác nhau có thể dễ dàng tương tác.

3.3 Ràng buộc, giả định và tiêu chí đánh giá

Để đảm bảo tính khả thi trong phạm vi đề án, hệ thống được thiết kế dựa trên các giới hạn rõ ràng:

1. **Ràng buộc:**

- *Về mặt kiến trúc:* Chọn mô hình Modular Monolith để giảm thiểu chi phí quản lý vận hành cho giai đoạn MVP, nhưng vẫn giữ tính phân tách cao để dễ dàng mở rộng.
- *Về mặt tài nguyên:* Offload toàn bộ tải tính toán AI nặng nề sang API của bên thứ ba, do đó hệ thống có thể chạy cục bộ trên phần cứng không yêu cầu GPU.
- *Về phạm vi hỗ trợ:* Ứng dụng trước hết là sẽ tập trung vào người dùng ở Việt Nam, thông qua đó xác định được các địa điểm lưu trú có mặt ở Việt Nam, ứng dụng phải hỗ trợ tiếng Việt và đảm bảo AI phải hiểu ngữ cảnh tiếng Việt.

2. Giả định:

- Khả năng lập luận, phân tích và đưa ra giải pháp của AI là yếu tố khách quan, không phụ thuộc hệ thống.
- Dữ liệu từ các nhà cung cấp bên ngoài đảm bảo độ tin cậy và thời gian phản hồi nhất định.

3. Tiêu chí đánh giá:

- *Sự hài lòng của người dùng (User Satisfaction):* Đánh giá mức độ thoải mái và hài lòng tổng thể của người dùng khi trải nghiệm các tính năng tìm kiếm và tương tác trên nền tảng.
- *Tính hữu ích và khả năng sử dụng (Usefulness & Usability):* Ứng dụng phải thực sự giải quyết được bài toán tìm chỗ ở, giao diện trực quan, dễ thao tác và luồng tương tác mượt mà cho mọi đối tượng.
- *Tính chính xác của thông tin (Information Accuracy):* Đảm bảo chất lượng và độ tin cậy của dữ liệu trả về, bao gồm độ sát thực tế của danh sách địa điểm gợi ý và tính chuẩn xác (không bịa đặt thông tin) trong các phân tích chi tiết từ LLM.

4 System Overview

SMACCO là một nền tảng tìm kiếm và quản lý lưu trú thông minh, kết hợp giữa hệ thống tra cứu truyền thống và sức mạnh của Trí tuệ Nhân tạo (AI). Dự án chuyển đổi cách người dùng tương tác với thông tin du lịch thông qua giao diện hội thoại (chatbot) và công nghệ tìm kiếm theo ngữ nghĩa (semantic search).

4.1 List of Core Features

- **Tìm kiếm và Gợi ý Lưu trú (Search & Recommendations):** Tìm kiếm đa chiều kết hợp bộ lọc truyền thống và gợi ý cá nhân hóa.
- **Trợ lý Ảo AI Tích hợp RAG (AI Chatbot & RAG):** Giao tiếp tự nhiên bằng tiếng Việt để tìm kiếm, phân tích và trích xuất thông tin thực tế từ hệ thống.
- **Quản lý Cộng đồng (Reviews & Questions):** Nền tảng chia sẻ đánh giá và hỏi đáp trực tiếp tại từng địa điểm lưu trú.
- **Theo dõi Hiện diện Tại chỗ (Presence / On-site Tracking):** Xác nhận và ưu tiên thông tin từ những người dùng đang có mặt tại địa điểm.
- **Đóng góp Nội dung (User Contributions & Uploads):** Cho phép người dùng tải lên hình ảnh, thêm mới hoặc cập nhật thông tin địa điểm.
- **Lưu trữ Địa điểm Yêu thích (Saved Places):** Đánh dấu và quản lý danh sách các địa điểm quan tâm.

4.2 Explanation of Core Features

- **Tìm kiếm và Gợi ý Lưu trú:** Hệ thống tổng hợp dữ liệu từ các API bản đồ (Goong, SerpAPI) và cơ sở dữ liệu nội bộ. Người dùng có thể tìm kiếm theo tọa độ, mức giá, tiện nghi, hoặc nhận các đề xuất được hệ thống tính toán dựa trên mức độ phù hợp.
- **Trợ lý Ảo AI Tích hợp RAG:** Sử dụng mô hình ngôn ngữ lớn (LLM) làm bộ định tuyến (router) để hiểu ý định người dùng (intent parsing). Khi người dùng đặt câu hỏi sâu về địa điểm, AI áp dụng kỹ thuật RAG (Retrieval-Augmented Generation) thông qua `pgvector` để trích xuất các bài đánh giá thực tế và tổng hợp thành câu trả lời chính xác, tránh hiện tượng sinh chữ ảo giác (hallucination).
- **Quản lý Cộng đồng:** Mỗi địa điểm lưu trú đóng vai trò như một diễn đàn thu nhỏ. Người dùng có thể đặt câu hỏi về tiện ích, chất lượng phục vụ và nhận câu trả lời từ những khách du lịch khác. Các bài đánh giá (reviews) này sau đó trở thành nguồn dữ liệu chất lượng cao cho AI.
- **Theo dõi Hiện diện Tại chỗ:** Điểm khác biệt cốt lõi là tính năng *check-in* tại chỗ. Hệ thống ghi nhận trạng thái “On-site” của người dùng, qua đó tăng độ tin cậy và ưu tiên hiển thị các câu trả lời/đánh giá từ những người đang thực sự trải nghiệm dịch vụ.

- **Đóng góp Nội dung:** Người dùng không chỉ là người tiêu thụ thông tin mà còn là người xây dựng hệ thống. Nền tảng cho phép tải lên hình ảnh thực tế và đề xuất chỉnh sửa thông tin chỗ ở, giúp dữ liệu luôn được làm mới.
- **Lưu trữ Địa điểm Yêu thích:** Người dùng có thể dễ dàng lưu lại các tùy chọn tiềm năng để so sánh và ra quyết định sau này.

4.3 Innovation Highlights

- **Tích hợp AI điều phối quy trình (AI-Driven Workflow):** AI không chỉ đóng vai trò sinh văn bản mà hoạt động như một cỗ máy điều phối (workflow engine). Thay vì click qua nhiều bộ lọc phức tạp, người dùng chỉ cần ra lệnh bằng tiếng Việt tự nhiên (ví dụ: “Tìm một homestay yên tĩnh ở Đà Lạt cho gia đình”), AI sẽ tự động gọi API, tính toán, và cập nhật kết quả thẳng lên giao diện (AI-driven UI).
- **Tìm kiếm Dựa trên Ngữ nghĩa (Semantic Search với Vector DB):** Việc sử dụng PostgreSQL kết hợp pgvector cho phép hệ thống hiểu được ý nghĩa sâu xa của các bài đánh giá. Thay vì so khớp từ khóa cứng nhắc, hệ thống có thể trả lời các câu hỏi trừu tượng dựa trên độ đo tương đồng Cosine (Cosine Similarity) của các vector nhúng (embeddings).
- **Ưu tiên Trải nghiệm Thực tế (On-site Insights):** Hạn chế rủi ro từ các đánh giá ảo bằng cách gán trọng số cao cho những tương tác của người dùng *On-site*. Điều này giải quyết bài toán nhức nhối trong ngành du lịch là sự sai lệch thông tin giữa quảng cáo và thực tế.

5 Pattern Recognition

6 Abstraction

7 System/Algorithm Design

7.1 Biểu diễn Bài toán

Khái niệm: Biểu diễn bài toán là quá trình chuyển đổi bài toán thực tế (xây dựng nền tảng khám phá chỗ ở thông minh) thành mô hình, cấu trúc dữ liệu và ký hiệu mà máy tính có thể xử lý.

Cách nhóm áp dụng 5 dạng biểu diễn:

1. **Biểu diễn dữ liệu:** Các thực thể thực tế (người dùng, địa điểm, đánh giá, hội thoại, tệp tin) được ánh xạ thành 13 mô hình quan hệ trong PostgreSQL. Mỗi thực thể được gán kiểu dữ liệu tương ứng: UUID cho định danh, JSONB cho dữ liệu linh hoạt, vector cho nhúng ngữ nghĩa.
2. **Biểu diễn cấu trúc:** Mỗi quan hệ giữa người dùng—địa điểm—nội dung được mô hình hóa qua ERD với khóa ngoại và chỉ mục. Sự phụ thuộc giữa các module được biểu diễn bằng sơ đồ lớp và C4 Model.
3. **Biểu diễn quy trình:** Luồng tương tác (tìm kiếm → gợi ý → trò chuyện AI → check-in) được mô hình hóa qua sơ đồ tuần tự và flowchart.
4. **Biểu diễn toán học:** Công thức tính điểm gợi ý đa yếu tố (rating, ngân sách, khoảng cách, tiện ích, cận kề) với trọng số và hàm chuẩn hóa.
5. **Biểu diễn logic:** Quy tắc định tuyến ý định người dùng (IF-THEN) qua LLM JSON Mode; logic kiểm tra điều kiện trong workflow registry.

Ba nguyên tắc cốt lõi được tuân thủ:

- **Trừu tượng hóa:** Chỉ lưu nội bộ địa điểm người dùng tương tác, ủy thác khám phá cho API ngoài — loại bỏ chi tiết không cần thiết.
- **Mô hình hóa có cấu trúc:** Mỗi module có ranh giới rõ ràng (controller/service/DTO), dữ liệu và quan hệ được đồng bộ qua Prisma schema.
- **Đa biểu diễn:** Cùng một bài toán (tìm kiếm chỗ ở) được biểu diễn đồng thời dưới dạng flowchart (luồng), công thức (điểm số) và mã giả (giải thuật hợp nhất).

7.2 Tổng quan Công nghệ

Hệ thống sử dụng ngăn xếp công nghệ thống nhất:

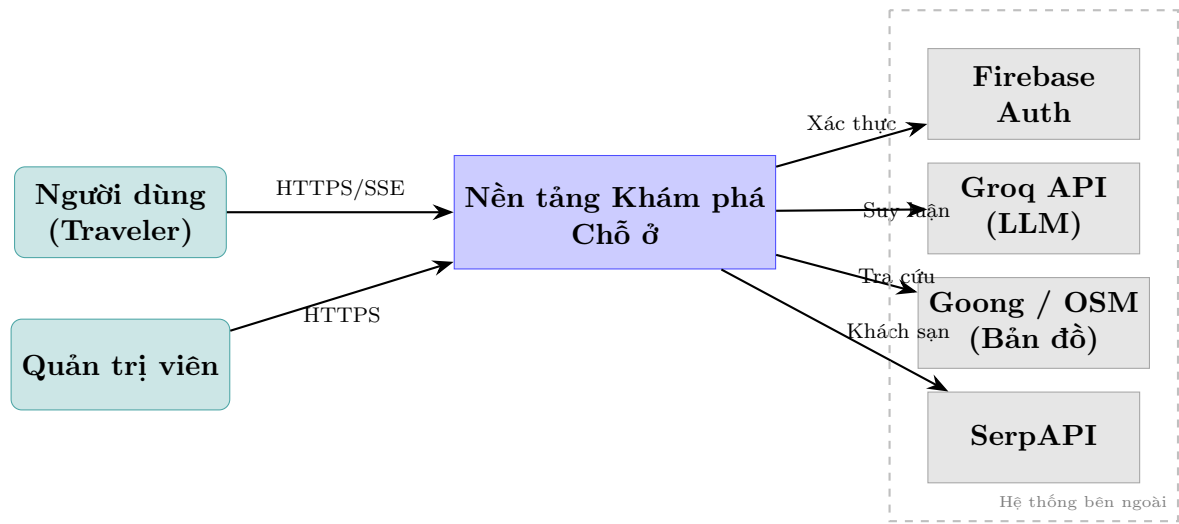
Tầng	Công nghệ	Mục đích
Giao diện Xác thực	React 18 + Vite 7 + TailwindCSS 3.4 Firebase Auth 10 (Web) + Admin SDK 12	SPA, bản đồ Leaflet, responsive Đăng nhập, JWT token
Backend ORM	NestJS 10 + TypeScript 5 Prisma 7.6	API REST, SSE streaming Truy xuất dữ liệu, migrations
Cơ sở dữ liệu	PostgreSQL 16 + pgvector	Dữ liệu quan hệ + vector nhúng
AI	Groq API (llama-3.1-70b)	Định tuyến ý định, tổng hợp phản hồi
Hạ tầng	Docker Compose V2 + Nginx 1.25	Triển khai, reverse proxy

Tích hợp bên ngoài: Goong API (tìm kiếm địa điểm chính), OpenStreetMap/Nominatim (mã hóa địa lý dự phòng), SerpAPI (truy vấn khách sạn).

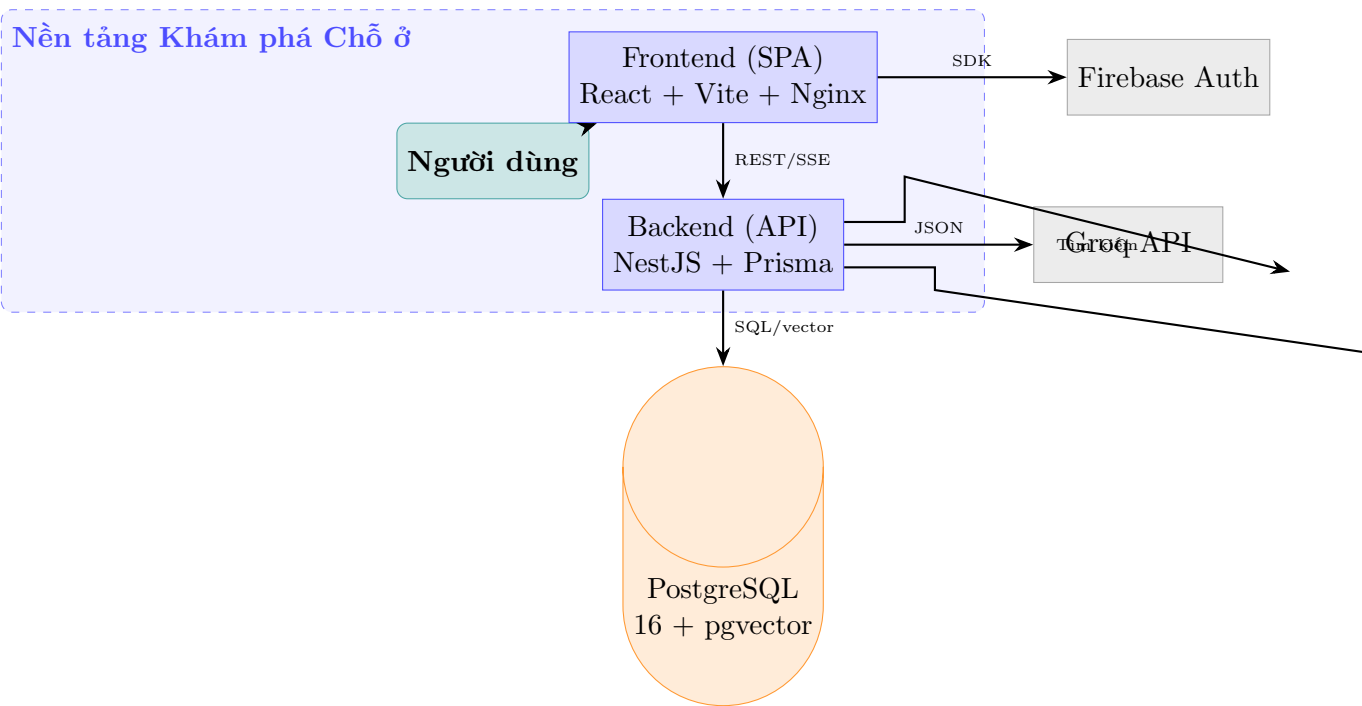
7.3 Mô hình C4

Mô hình C4 được sử dụng để biểu diễn kiến trúc hệ thống ở bốn cấp độ, mỗi cấp độ phục vụ một đối tượng người xem khác nhau.

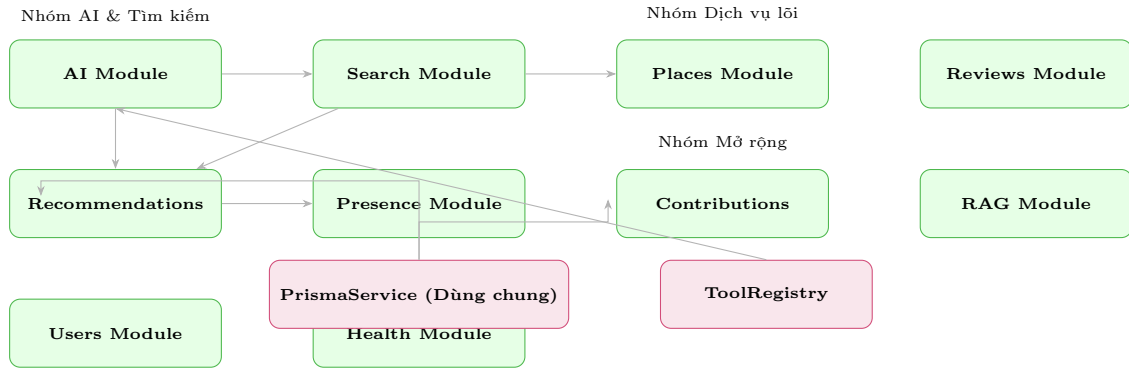
7.3.1 Cấp 1 — System Context



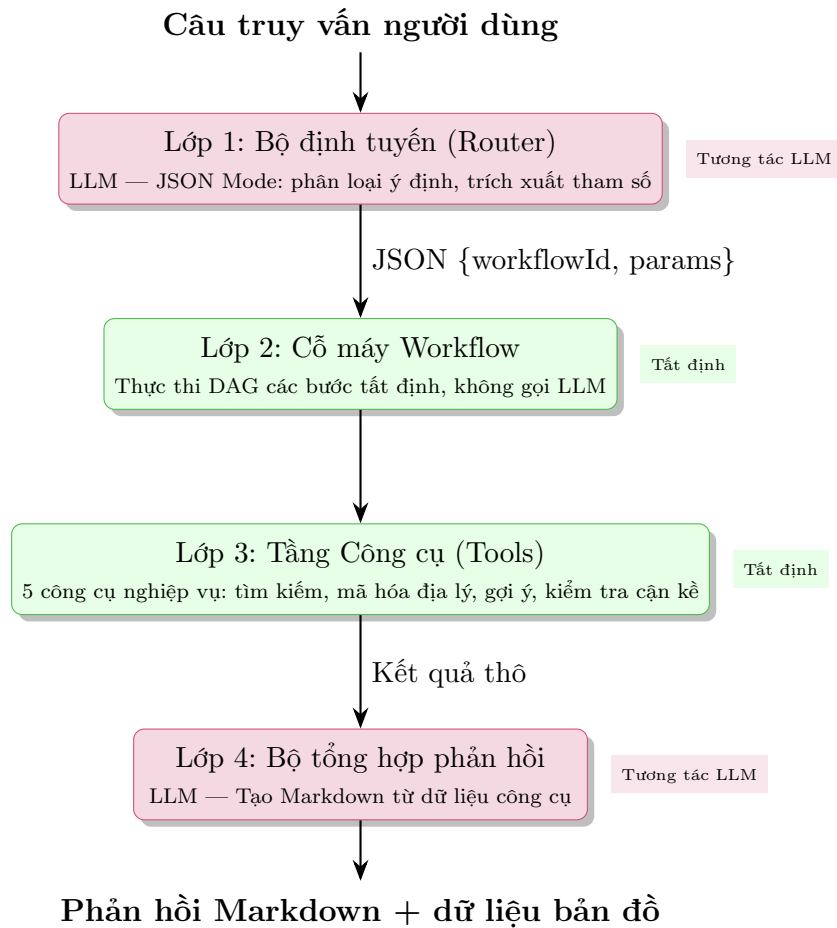
7.3.2 Cấp 2 — Container



7.3.3 Cấp 3 — Component



7.3.4 Cấp 4 — Code (Chi tiết Module AI)

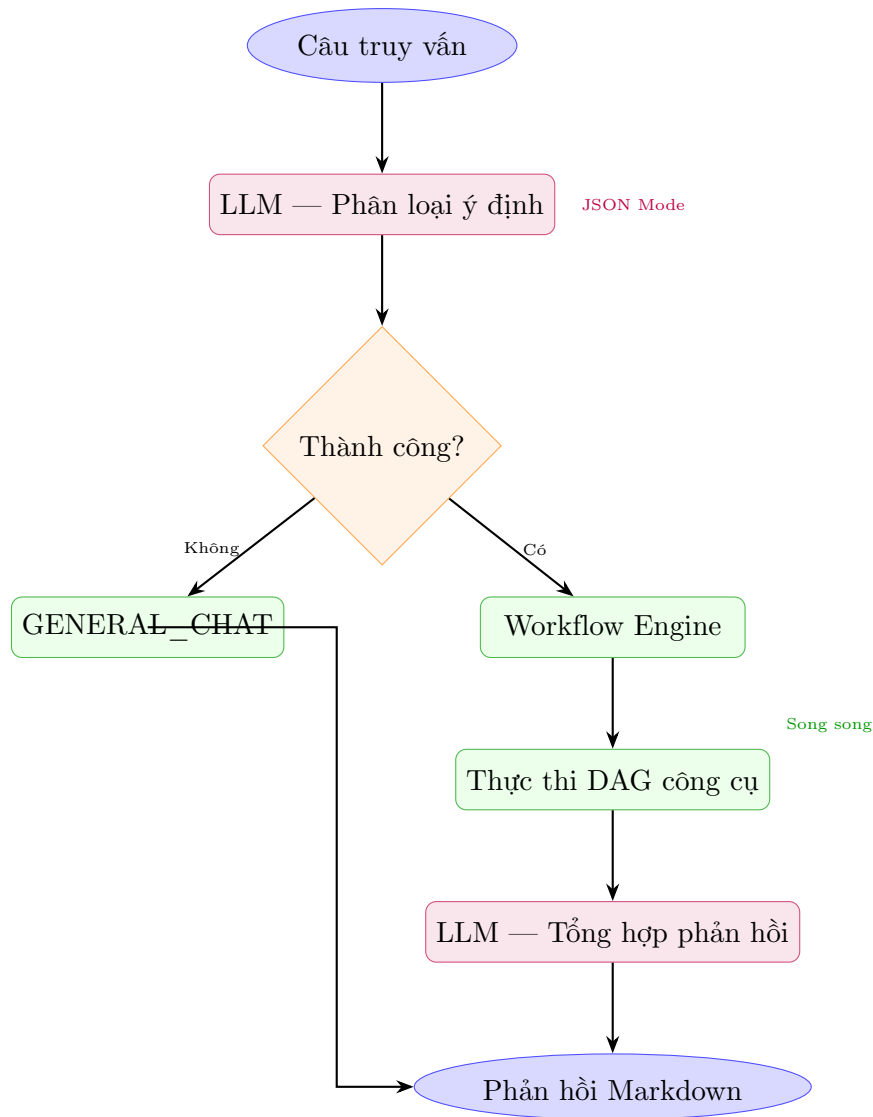


7.4 Thiết kế Giải thuật theo Chức năng

7.4.1 Điều phối AI

Mô tả: Giải thuật điều phối AI chuyển đổi câu truy vấn người dùng thành phản hồi có cấu trúc thông qua kiến trúc bốn lớp.

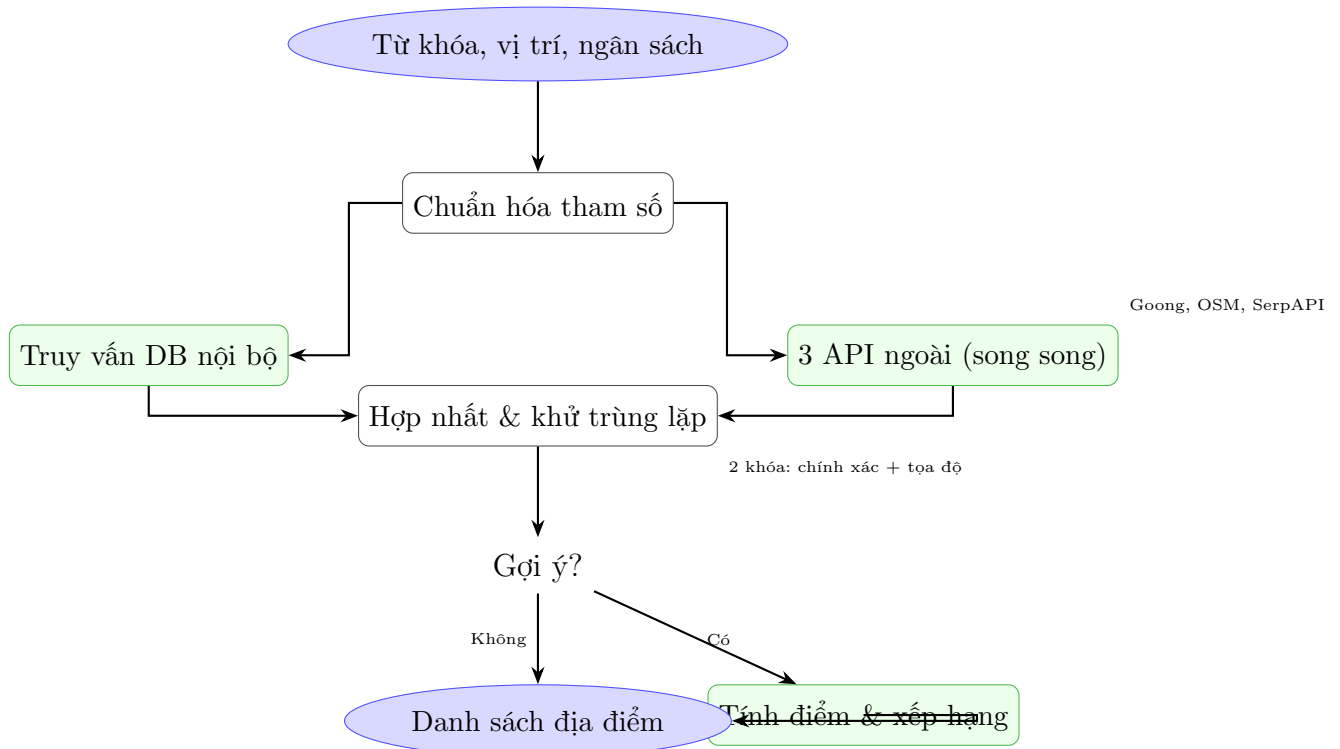
Đầu vào: Câu truy vấn tiếng Việt, lịch sử hội thoại. **Đầu ra:** Phản hồi Markdown kèm dữ liệu bản đồ (searchAction).



Đánh giá: Độ phức tạp $O(|DAG| + |LLM|)$ với $|DAG| \leq 3$. Hai trong bốn lớp không tương tác LLM, đảm bảo tính tất định cho logic nghiệp vụ. Cơ chế an toàn: Router thất bại $\rightarrow$ GENERAL_CHAT; Tool lỗi $\rightarrow$ đánh dấu và tiếp tục.

7.4.2 Tìm kiếm Lai (Hybrid Search)

Mô tả: Kết hợp dữ liệu từ cơ sở dữ liệu nội bộ và ba nhà cung cấp ngoài (Goong, OSM, SerpAPI) để tạo danh sách địa điểm toàn diện.



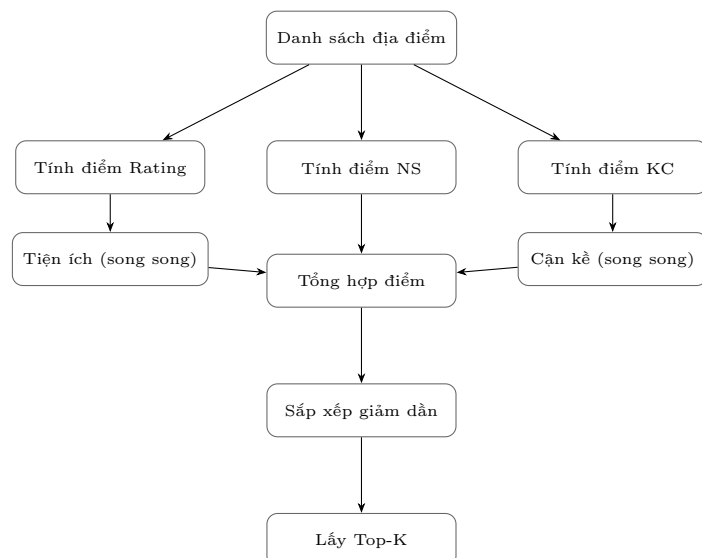
Đánh giá: Độ phức tạp $O(n + m)$. Cách ly lỗi qua catch riêng biệt — một nhà cung cấp thất bại không ảnh hưởng đến các nhà cung cấp khác. Chuỗi dự phòng: Goong → OSM → mảng rỗng.

7.4.3 Gợi ý Xếp hạng

Mô tả: Chuyển đổi kết quả tìm kiếm thô thành danh sách xếp hạng sử dụng mô hình đa yếu tố.

Công thức tính điểm:

$$\text{Điểm} = 0,30 \times \text{Rating} + 0,20 \times \text{Ngân_sách} + 0,20 \times \text{Khoảng_cách} + 0,15 \times \text{Tiện_ích} + 0,15 \times \text{Cận_kề}$$



Khi không có vị trí neo, trọng số khoảng cách và cận kề được phân phối lại cho các yếu tố còn lại. Địa điểm có người đang hiện diện nhận điểm thưởng.

7.4.4 Đường ống RAG

Pipeline phục vụ tìm kiếm ngữ nghĩa trên dữ liệu đóng góp (đánh giá, tệp tin):

- **Tiếp nhận:** Nguồn (review/file) → Phân đoạn văn bản → Sinh embedding → Lưu vào bảng **chunks** với vector.
- **Truy xuất:** Truy vấn → Sinh embedding → Tìm kiếm tương đồng cosine (pgvector) → Xây dựng ngữ cảnh → LLM.

Trạng thái hiện tại: Lược đồ và lưu trữ chunk đã hoàn thiện; sinh embedding đang chờ triển khai.

7.4.5 Các Giải thuật Bổ trợ

- **Theo dõi hiện diện (Presence):** Check-in/check-out với 3 endpoint. Dữ liệu hiện diện phục vụ UI thời gian thực và làm yếu tố tăng điểm trong gợi ý.
- **Bỏ phiếu Q&A:** Mô hình Reddit với ràng buộc UNIQUE(answer_id, user_id). Giá trị vote (1/-1) được vật chất hóa trên bảng answers.
- **Khử trùng lặp:** Chiến lược hai khóa (chính xác: source:source_place_id; mờ: tên_chuẩn_hóa:lat(3):lng(3)) đảm bảo toàn vẹn khi hợp nhất đa nguồn.

7.5 Đánh giá Hiệu năng và Khả năng Mở rộng

Nút thắt	Mức rủi ro	Biện pháp giảm thiểu
Độ trễ LLM (700ms+)	Cao	Streaming, thực thi công cụ song song trước
Giới hạn tốc độ API ngoài	Trung bình	Chuỗi dự phòng, cache
Tìm kiếm vector tập lớn	Trung bình	Chỉ mục IVFFlat
Truy vấn N+1 Prisma	Thấp	Eager loading

8 Implementation

8.1 Từ Mô hình Thiết kế đến Phân rã Module

Từ các mô hình đã xây dựng ở phần Thiết kế Hệ thống (C4 Model, ERD, sơ đồ tuần tự), nhóm tiến hành phân rã thành các module triển khai theo ba bước:

1. **Phân rã chức năng:** Các khối chức năng trong C4 Container → 10 module NestJS, mỗi module gồm tầng giao tiếp (API), tầng logic (xử lý nghiệp vụ) và tầng dữ liệu (DTO).
2. **Ánh xạ dữ liệu:** Mô hình ERD → 13 lược đồ Prisma với quan hệ, ràng buộc và chỉ mục. Các thao tác vector nhúng (pgvector) được xử lý qua SQL thô do hạn chế của ORM.
3. **Ánh xạ quy trình:** Sơ đồ tuần tự → workflow registry dạng JSON và các lớp điều phối AI. Mỗi ca sử dụng tương ứng với một workflow cụ thể trong registry.

Các module được tổ chức thành bốn nhóm:

Nhóm	Module	Nguồn gốc
Dịch vụ lõi	Users, Places, Reviews, Search, Health	Kế thừa từ core-service (NestJS)
AI & Tìm kiếm	AI, Recommendations	Chuyển đổi từ FastAPI sang NestJS
Mở rộng MVP	RAG, Presence, Contributions	Phát triển mới
Dùng chung	PrismaService, ToolRegistry, Global Filters	Toàn cục, dùng chung qua DI

8.2 Di trú Kiến trúc: Microservices → Modular Monolith

Hệ thống nguyên bản vận hành với bốn dịch vụ riêng rẽ (Nginx Gateway, Core-service, AI-service, Recommendation-service). Kiến trúc này bộc lộ các vấn đề:

- **Chi phí mạng:** Mỗi yêu cầu chat trải qua ba chặng HTTP, gây độ trễ tích lũy.
- **Trùng lặp logic:** Xác thực, xử lý lỗi, cấu hình bị sao chép giữa các dịch vụ.
- **Phân mảnh kiểu:** Python ↔ TypeScript yêu cầu tuần tự hóa thủ công, dễ sai sót.
- **Phức tạp vận hành:** Bốn Docker image, bốn pipeline triển khai riêng biệt.

Giải pháp: Hợp nhất thành một monolith dạng module NestJS duy nhất. Các lời gọi HTTP liên dịch vụ được thay thế bằng phương thức gọi trực tiếp qua dependency injection, loại bỏ Gateway Nginx trung gian.

8.3 Các Thách thức và Giải pháp

8.3.1 Tính phi tất định của LLM

Thách thức: LLM sinh phản hồi không nhất quán về định dạng, dễ hallucination dữ liệu, và độ trễ khó dự đoán.

Giải pháp — Kiến trúc Bốn Lớp:

- **Lớp 1 (Router):** LLM với JSON Mode → phân loại ý định, trích xuất tham số có cấu trúc. Thất bại → mặc định GENERAL_CHAT.
- **Lớp 2 (Workflow Engine):** Thực thi DAG các bước tất định, không gọi LLM.
- **Lớp 3 (Tool Layer):** Mỗi công cụ có hợp đồng đầu ra chuẩn hóa, không ném ngoại lệ.
- **Lớp 4 (Composer):** Neo phản hồi Markdown vào dữ liệu thực tế từ công cụ, giảm hallucination.

Lợi ích: Workflow mới có thể thêm dưới dạng JSON, không cần biên dịch lại.

8.3.2 Streaming SSE qua POST

Thách thức: SSE chuẩn chỉ hỗ trợ GET, trong khi endpoint chat cần gửi JSON payload và token xác thực.

Giải pháp — Mô hình hai pha:

- Sử dụng `fetch()` với `ReadableStream` thay vì `EventSource`, cho phép POST và đính kèm token.
- Pha 1: Gửi dữ liệu cấu trúc (`searchAction`) ngay sau khi công cụ thực thi — cho phép frontend render bản đồ trước.
- Pha 2: Truyền token LLM dưới dạng delta SSE.
- Header `X-Accel-Buffering`: no để vô hiệu hóa bộ đệm Nginx khi triển khai sau reverse proxy.

8.3.3 Tích hợp Đa Nhà cung cấp Bản đồ

Thách thức: Mỗi API bản đồ trả về định dạng khác nhau; lỗi một nhà cung cấp không được ảnh hưởng toàn bộ tìm kiếm.

Giải pháp:

- Chuẩn hóa giao diện đầu ra cho mọi nhà cung cấp.
- Gọi song song qua `Promise.all` với `.catch()` riêng biệt — đảm bảo cách ly lỗi.
- Chuỗi dự phòng: Goong → OSM → mảng rỗng.

8.3.4 Vector Embedding và Prisma

Thách thức: Prisma chưa hỗ trợ nguyên sinh kiểu `vector` của PostgreSQL.

Giải pháp: Khai báo cột `embedding` dưới dạng `Unsupported("vector")`, thao tác qua SQL thô (`$queryRawUnsafe`).

8.3.5 Xác thực Firebase

Thách thức: Tích hợp Firebase JWT vào vòng đời NestJS guard mà không sao chép logic xác thực trên mọi controller.

Giải pháp: Triển khai guard với `CanActivate`: trích xuất `Authorization` header → xác thực token qua Firebase Admin SDK → gắn decoded token vào request. Frontend Axios interceptor tự động đính kèm token.

8.4 Các Thách thức Khác

Xử lý đồng thời: Ba mẫu thiết kế được áp dụng: (1) `Promise.all` với catch riêng biệt; (2) ghi vào khóa khác nhau trong shared state để tránh race condition; (3) Active Flag pattern để tránh cập nhật state sau khi component unmount.

Triển khai Docker: Ba container (frontend/Nginx, backend/Node 22, PostgreSQL/pgvector) với chiến lược build đa tầng tận dụng layer cache. Docker override cho môi trường phát triển sử dụng volume ẩn danh tránh xung đột `node_modules`.

Quản lý bản đồ frontend: Chuyển đổi layer bằng opacity, MarkerClusterGroup với chunkedLoading cho hiệu năng, CustomEvent thay vì prop drilling cho tích hợp AI-Search.

8.5 Kết luận

Quá trình triển khai cho thấy việc chuyển đổi từ mô hình thiết kế sang mã nguồn đòi hỏi liên tục giải quyết các vấn đề thực tế: tính phi tất định của LLM, hạn chế của ORM với kiểu dữ liệu đặc thù, và đảm bảo tính nhất quán xuyên suốt các tầng. Mười hai vấn đề đã được ghi nhận với các mức độ ưu tiên khác nhau.

9 Testing

9.1 Chiến lược Kiểm thử

Nhóm áp dụng chiến lược kiểm thử đa tầng, bao phủ từ đơn vị đến tích hợp, với mục tiêu đảm bảo chất lượng cho toàn bộ hệ thống:



Nguyên tắc: Mock toàn bộ tầng dữ liệu và dịch vụ ngoài ở unit test; kiểm thử tích hợp sử dụng module thật với dependency injection đầy đủ. Mỗi tầng kiểm thử bổ sung và mở rộng cho tầng trước.

9.2 Phạm vi Kiểm thử

9.2.1 Frontend — Giao diện và Tương tác

- **Dịch vụ (Service layer):** Kiểm thử 12 dịch vụ frontend bao gồm API chat, tìm kiếm, địa điểm, đánh giá, hiện diện, đóng góp tệp, gợi ý, RAG, người dùng — xác nhận mỗi service gọi đúng endpoint, xử lý đúng dữ liệu trả về và bắt lỗi HTTP.
- **Context:** Kiểm thử 3 context (Auth, TravelData, Theme) — xác nhận provider render đúng, cung cấp giá trị mặc định và cập nhật state khi có thay đổi.
- **Hooks:** Kiểm thử hook streaming AI — ghép chunk delta SSE, xây dựng prompt ngữ cảnh, xử lý lỗi luồng. Kiểm thử ánh xạ lỗi Firebase — dịch 4 mã lỗi (permission-denied, unauthenticated, network-request-failed, unknown) sang thông báo tiếng Việt.
- **Component:** Kiểm thử 25 component giao diện chính: MapComponent (hiển thị marker, cluster), ChatPanel (gửi/nhận tin nhắn, streaming), PlaceCard (hiển thị thông tin, nút tương tác), SearchBar (tìm kiếm, gợi ý), các component wizard-inputs (LocationPicker, BudgetSelector, TypeSelector) và chat subcomponents (MessageList, StreamingText).
- **Trang:** Kiểm thử 5 trang: HomePage, SearchPage, PlaceDetailPage, ProfilePage, AdminPage — xác nhận layout, điều hướng và tích hợp component.

9.2.2 Backend — Xử lý Nghiệp vụ và Tích hợp

- **Dịch vụ (Services):** Kiểm thử 17 service backend bao phủ toàn bộ module: Người dùng (CRUD, upsert, tìm kiếm), Địa điểm (CRUD, lọc, fuzzy match), Đánh giá

(CRUD, điểm số), Tìm kiếm (đa nhà cung cấp, hợp nhất, khử trùng lặp), Gợi ý (tính điểm, xếp hạng, trọng số), Hiện diện (join/leave/get, tải đồng thời), Đóng góp (tạo/lấy/cập nhật tệp), RAG (lưu/truy xuất chunk, xây dựng ngữ cảnh), Hội thoại AI (danh sách/tạo/xóa/tin nhắn), NLP (trích xuất bộ lọc từ văn bản tiếng Việt).

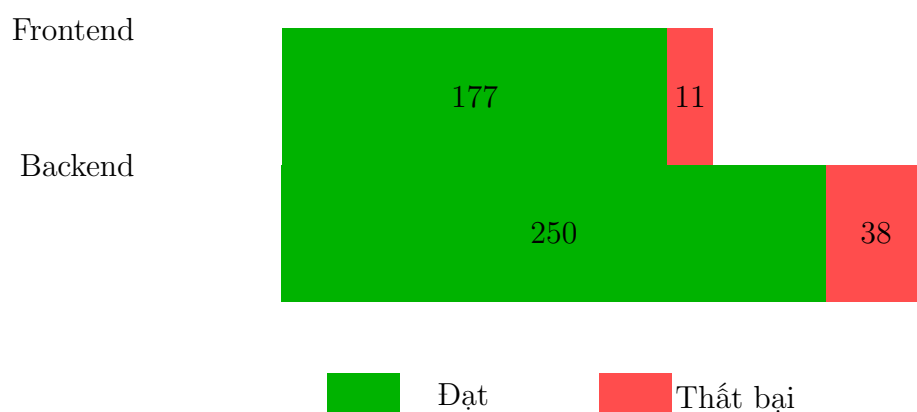
- **Controller:** Kiểm thử 12 controller — xác nhận route đăng ký đúng, validation pipe hoạt động, response format chuẩn.
- **Công cụ AI (Tools):** Kiểm thử 10 tool — SearchPlaces (tìm kiếm lại), RecommendPlaces (xếp hạng), GeocodeAnchor (mã hóa địa lý), ProximityChecker (khoảng cách), NearbyAmenities (tiện ích lân cận), và các tool bổ trợ. Mỗi tool được kiểm tra cả luồng thành công và thất bại.
- **Guard/Filter/Interceptor:** Kiểm thử FirebaseAuthGuard (thiếu header, sai định dạng, token không hợp lệ), AllExceptionsFilter (HttpException, lỗi không xác định), LoggingInterceptor (định nghĩa, phương thức).

9.2.3 Các Tình huống và Giả lập Đặc thù

- **Giả lập lỗi nhà cung cấp ngoài:** Khi Goong API thất bại, hệ thống tự động chuyển sang OSM; khi cả hai thất bại, trả về kết quả rỗng an toàn. Một nhà cung cấp lỗi không ảnh hưởng đến kết quả từ các nhà cung cấp khác.
- **Xử lý workflow thất bại:** Router phân tích lỗi → dự phòng GENERAL_CHAT; tool lỗi → đánh dấu lỗi và tiếp tục thực thi workflow.
- **SSE streaming đầu cuối:** Xác minh kết nối, nhận chunk delta, xử lý finish, buffer ghép mảnh.
- **Dữ liệu rỗng và trường hợp biên:** Tìm kiếm không kết quả, hội thoại mới, người dùng chưa có profile, địa điểm không có đánh giá.
- **Tải đồng thời:** 100 người dùng join cùng lúc vào cùng địa điểm — kiểm tra tính toàn vẹn của cấu trúc dữ liệu.

9.3 Kết quả Kiểm thử

Kết quả kiểm thử tổng thể được thống kê như sau:



Frontend — 177/188 đạt (94,1%)

12 dịch vụ, 3 context, 3 hooks, 5 trang, 25 component

Backend — 250/288 đạt (86,8%)

17 services, 12 controllers, 10 tools, 3 guard/filter/interceptor

Tổng thể — 427/476 đạt (89,7%)

Phân bố kiểm thử theo nhóm chức năng backend:



Phân bố kiểm thử theo nhóm chức năng frontend:

Nhóm chức năng	Số dịch vụ/Component	Kết quả
Dịch vụ API (aiService, searchService, placeService, ...)	12 services	11/12
Ngữ cảnh (AuthContext, TravelDataContext, ThemeContext)	3 contexts	2/3
Hooks (streamingChat, firestoreError)	3 hooks	3/3
Component giao diện (Map, Chat, PlaceCard, SearchBar, ...)	25 components	25/25
Trang (Home, Search, PlaceDetail, Profile, Admin)	5 pages	5/5

Ghi chú: 11 ca thất bại frontend và 38 ca thất bại backend đều liên quan đến mock Firebase/Firestore chưa hoàn chỉnh — không phải lỗi logic nghiệp vụ. Tất cả module chức năng chính đều được kiểm thử thành công.

9.4 Cải tiến Sau Kiểm thử

Quá trình kiểm thử đã phát hiện và dẫn đến các cải tiến sau:

- **Cải tiến xử lý lỗi:** Chuẩn hóa phản hồi lỗi thành cấu trúc JSON thống nhất với timestamp, loại bỏ try/catch trùng lặp ở controller.
- **Cải tiến khử trùng lặp:** Bổ sung khóa mờ theo tọa độ (lat:lng độ chính xác 3 chữ số thập phân) bên cạnh khóa chính xác, giúp phát hiện trùng lặp khi ID nhà cung cấp khác nhau.
- **Cải tiến buffer SSE:** Cả backend và frontend được đồng bộ buffer pattern để xử lý chunk SSE bị phân mảnh do TCP.
- **Cải tiến Presence:** Thêm kiểm tra tải 100 người dùng đồng thời, đảm bảo cấu trúc dữ liệu hoạt động ổn định dưới tải cao.
- **Cải tiến NLP:** Tham số hóa 14 văn bản tiếng Việt thực tế, bao phủ đa dạng biến thể ngôn ngữ (tên tỉnh thành, loại hình, mức giá).

9.5 Hướng Phát triển

Các khoảng trống kiểm thử đã được xác định:

- Kiểm thử tích hợp với PostgreSQL + pgvector thực (hiện đang mock Prisma).
- Pipeline CI/CD tự động hóa quy trình kiểm thử.
- Kiểm thử hiệu năng với tập dữ liệu lớn (10^5 + địa điểm).
- Kiểm thử người dùng (usability testing) có tổ chức.

10 Demo

11 Deployment

12 Logbook: Plan and Actual Workload

13 AI usage and declaration

14 Kết luận và hướng phát triển

14.1 Kết luận

Trong quá trình thực hiện đề tài, nhóm đã xây dựng thành công ứng dụng web toàn diện SMACCO với kiến trúc hiện đại, tích hợp các công nghệ AI tiên tiến. Hệ thống được phát triển trên nền tảng React (frontend) và NestJS (backend), kết hợp với Prisma ORM, Firebase Authentication, và các dịch vụ AI đa nền tảng (Gemini, Cloudflare AI, FreeModel).

Các chức năng đã được triển khai thành công bao gồm:

- Chức năng tìm kiếm qua chatbot
- Chức năng so sánh các địa điểm lưu trú
- Chức năng trích xuất thông tin chi tiết và đánh giá của AI về một địa điểm lưu trú cụ thể

14.2 Những kiến thức và kỹ năng đạt được

Môn học và đồ án lần này đã mang lại cho chúng em những bài học vô cùng thực tế, giúp thu hẹp khoảng cách giữa lý thuyết và ứng dụng:

- **Vận dụng 4 trụ cột cốt lõi của Tư duy Máy tính (Computational Thinking):** Thông qua đồ án, chúng em đã làm chủ và áp dụng thành công mô hình tư duy này để giải quyết bài toán thực tế. Cụ thể, chúng em thực hiện
 1. *Phân rã bài toán* (Decomposition) để chia nhỏ hệ thống thành các module độc lập (*AiModule* và *RecommendationsModule*);
 2. *Nhận biết mẫu* (Pattern Recognition) để tìm ra khuôn mẫu trong hành vi tìm kiếm của khách hàng;
 3. *Trừu tượng hóa* (Abstraction) để lọc bỏ thông tin nhiễu, trích xuất tham số cốt lõi qua *Intent Parsing*;
 4. *Thiết kế thuật toán* (Algorithm Design) để xây dựng luồng vận hành (Workflow) tối ưu cho Chatbot.

Các kỹ năng này giúp chúng em định hình được tư duy logic vững chắc cho các dự án phức tạp sau này.

- **Tư duy khai thác Trí tuệ nhân tạo:** Chúng em đã học được cách viết cấu trúc câu lệnh (Prompt Engineering) và cách ứng dụng AI một cách hiệu quả, biến AI thành một công cụ đồng hành đắc lực để tối ưu hóa việc tìm kiếm thông tin thay vì lạm dụng một cách máy móc.
- **Làm quen với các công nghệ mới:** Đồ án là cơ hội để chúng em tiếp cận và biết cách sử dụng hàng loạt công cụ, ngôn ngữ và nền tảng cốt lõi trong phát triển phần mềm hiện đại như HTML, JavaScript, nền tảng cơ sở dữ liệu Firebase, bản đồ OpenStreetMap,... Dù thời gian có hạn và chưa thể tìm hiểu quá sâu sắc từng công nghệ, nhưng đây là nền móng cực kỳ quan trọng cho các môn học chuyên ngành tiếp theo.

- **Kỹ năng làm việc nhóm và Quản lý mã nguồn:** Biết cách sử dụng GitHub để quản lý các phiên bản code, phối hợp phân chia công việc trong nhóm một cách hệ thống.
- **Tư duy Hệ thống:** Hiểu rõ bức tranh toàn cảnh về cách một hệ thống phần mềm vận hành trong thực tế (cách Frontend tương tác với Backend, cách gọi API và truy vấn dữ liệu từ Database).

14.3 Hướng phát triển trong tương lai

Nhằm nâng cấp và hoàn thiện dự án thành một sản phẩm có tính ứng dụng thực tế cao hơn, các mục tiêu trong tương lai bao gồm:

- **Nghiên cứu sâu về RAG và LLM:** Tối ưu hóa sâu hơn kỹ thuật Retrieval-Augmented Generation (RAG) kết hợp với các mô hình ngôn ngữ lớn (như Groq) để nâng cao độ chính xác, giảm thiểu hiện tượng "ảo tưởng" (hallucination) của Chatbot khi trả lời khách hàng.
- **Tối ưu hóa UI/UX:** Thiết kế lại giao diện người dùng thân thiện, trực quan hơn trên cả hai nền tảng Web và Mobile, tối ưu hóa các bộ lọc nâng cao để nâng cao trải nghiệm của các nhóm đối tượng Target Users.